Báo cáo chi tiết seminar — cập nhật đến Tuần 06

Nhóm: Group09. **Phạm vi:** tổng hợp nội dung seminar và minh chứng tích lũy; báo cáo tiến độ theo tuần nằm ở `BangCongViec.md` và `evidence/tuan-06/README.md`. **Cập nhật tài liệu:** 28/10/2026 (bổ sung phần Ngô Gia An). Tài liệu được biên tập với Codex hỗ trợ và cần nhóm review trước khi trình bày.

Thành viên: TV1: Phạm Quang Vinh (23120202); TV2: Nguyễn Bảo An (23120207); TV3: Trương Nhật Đạt (23120231); TV4: Nguyễn Lê Thế Vinh (23120190); TV5: Ngô Gia An (23120205)

1. Mục tiêu và phạm vi

Nhóm em tìm hiểu chuỗi Unit Test → Code Coverage → CI/CD trên một ứng dụng quản lý thư viện. Người học cần viết được test cho logic có điều kiện, đọc được phần mã chưa được phủ và hiểu cách đưa test/coverage vào pipeline kiểm tra trước khi tích hợp mã.

Case study gồm tính phí trả sách trễ, mượn sách qua ba repository và API `POST /api/loans`. Dữ liệu nằm trong H2 in-memory qua Spring Data JPA (dữ liệu mẫu trong `data.sql`), nên mọi người chạy lại được mà không cần cài database. Công nghệ theo thiết kế gốc: Java 25, Spring Boot 4.1.1, JUnit Jupiter 6. Cấu hình CI thực hiện kiểm thử và đóng gói, chưa triển khai lên môi trường sử dụng thật.

2. Unit Test

2.1. Khái niệm tổng quan về Unit Testing

Unit test (kiểm thử đơn vị) kiểm tra một đơn vị phần mềm nhỏ nhất có thể cô lập được trong hệ thống (thường là một hàm, phương thức hoặc một lớp) với tập đầu vào và đầu ra xác định.

- **Định nghĩa Unit:** Một "unit" không nhất thiết chỉ là một hàm đơn lẻ mà là một đơn vị hành vi (unit of behavior). Trong kiến trúc hướng đối tượng, đơn vị này có thể là một class độc lập hoặc một cụm đối tượng phối hợp chặt chẽ, được kiểm thử cách ly khỏi các phụ thuộc bên ngoài (cơ sở dữ liệu, mạng, hệ thống file).
- **Quy tắc AAA (Arrange - Act - Assert):**
    - *Arrange:* Chuẩn bị dữ liệu đầu vào, cấu hình trạng thái ban đầu và thiết lập các phụ thuộc giả lập (Test Double).
    - *Act:* Kích hoạt thực thi hành vi hoặc phương thức cần kiểm thử với dữ liệu đã chuẩn bị.
    - *Assert:* Kiểm tra và khẳng định kết quả thực tế trả về hoặc tác dụng phụ (side-effect) phải khớp hoàn toàn với kỳ vọng.
- **Nguyên tắc FIRST:**
    - *Fast (Nhanh):* Test phải chạy trong mili-giây để lập trình viên có thể chạy liên tục sau mỗi thay đổi nhỏ mà không ngại độ trễ.
    - *Independent (Độc lập):* Không test case nào phụ thuộc vào kết quả hoặc thứ tự chạy của test case khác.
    - *Repeatable (Lặp lại được):* Chạy 1.000 lần ở bất kỳ môi trường nào (máy cá nhân, CI server, không có mạng) đều cho ra cùng một kết quả.

    - *Self-validating (Tự xác minh):* Test tự động trả về Pass hoặc Fail rõ ràng thông qua assertion, không đòi hỏi con người đọc log hay file để đoán kết quả.
    - *Timely (Kịp thời):* Được viết song song hoặc ngay trước khi viết code sản phẩm (theo phong cách TDD).

```
      /\
     /  \        UI / End-to-End Tests (Ít, Chậm, Chi phí cao)
    /----\
   /      \      Integration / Service Tests (Trung bình, phụ thuộc component)
  /--------\
 /          \    Unit Tests (Nhiều nhất, Nhanh nhất, Chi phí thấp nhất)
/------------\
```

- **Kim tự tháp kiểm thử (Test Pyramid):** Mô hình do Mike Cohn đề xuất chỉ ra rằng Unit Test phải chiếm phần đáy lớn nhất của kim tự tháp (70-80% tổng số test) vì tính chất thực thi siêu nhanh, chi phí bảo trì thấp và khả năng định vị lỗi chính xác ở cấp độ dòng lệnh.
- **Phân loại Test Double (Gerard Meszaros):** Khi kiểm thử một đơn vị có phụ thuộc phức tạp, ta dùng các đối tượng thay thế:
    1. *Dummy:* Đối tượng truyền vào chỉ để thỏa mãn chữ ký phương thức (tham số bắt buộc), không bao giờ được gọi hay sử dụng logic bên trong.
    2. *Stub:* Cung cấp sẵn các câu trả lời cố định (canned answers) cho các lệnh gọi trong quá trình test, không quan tâm đối tượng được gọi như thế nào.
    3. *Fake:* Đối tượng có cài đặt hoạt động thực sự nhưng dùng đường tắt để đơn giản hóa, không phù hợp cho môi trường production (ví dụ: Database In-Memory H2 thay vì Oracle/PostgreSQL thật).
    4. *Mock:* Đối tượng được lập trình trước với các kỳ vọng cụ thể về hành vi (expectations): phương thức nào sẽ được gọi, bao nhiêu lần, với tham số gì, và sẽ verify lại sau khi Act.
    5. *Spy:* Đối tượng bọc lấy đối tượng thật (wrapper) để vừa thực thi logic thật, vừa ghi nhận/theo dõi lại các tương tác (số lần gọi, tham số).
- **Khái niệm Test Oracle:** Test Oracle là cơ chế hoặc nguồn chân lý (source of truth) dùng để xác định xem kết quả thực thi của phần mềm là đúng hay sai. Trong Unit Test, Oracle có thể là:
    - Giá trị tính toán kỳ vọng định sẵn từ đặc tả nghiệp vụ.
    - Một mô hình toán học hoặc thuật toán tham chiếu độc lập.
    - Dữ liệu lịch sử đã được kiểm chứng (Golden Master).

2.2. Kỹ thuật áp dụng trong Case Study

Với `FineCalculator`, dùng đối tượng thật vì lớp chỉ chứa logic. Với `LoanService`, dùng mock cho ba repository để tập trung vào quyết định nghiệp vụ. Stub cung cấp phản hồi từ phụ thuộc; mock/verify kiểm tra cách tương tác. `Clock.fixed` loại bỏ phụ thuộc giờ máy. Không dùng mock chính đối tượng đang cần kiểm logic.

2.3. Phân tích ba kỹ thuật kiểm thử cốt lõi

1. **Data-driven Testing (Kiểm thử hướng dữ liệu):**
    - *Bản chất:* Tách biệt hoàn toàn giữa mã lệnh thực thi kiểm thử và ma trận dữ liệu kiểm thử (test input & expected output). Khi thay đổi yêu cầu hoặc bổ sung kịch bản kiểm thử biên, lập trình viên chỉ cần thêm dữ liệu vào bảng mà không phải sửa logic mã test.

- *Minh chứng trong Case Study:* Dự án sử dụng `@ParameterizedTest` kết hợp `@CsvFileSource(resources = "/fine-cases.csv")` trong `FineCalculatorTest`. Tập dữ liệu 15 dòng mô tả đầy đủ các biên phân lớp tương đương: ngưỡng ân hạn của NORMAL (0 ngày, 1 ngày) và VIP (0, 1, 2, 3 ngày), các mốc tiền tính theo ngày, và giá trị biên chạm trần 50.000đ (NORMAL 25 ngày, VIP 52 ngày).

2. **Checkpoints (Điểm kiểm tra):**
   - *Bản chất:* Checkpoint là các mốc được đặt xuyên suốt quy trình kiểm thử để kiểm tra tính hợp lệ của hệ thống tại từng thời điểm. Trong Unit Test, checkpoint thể hiện ở hai khía cạnh:
     - *State Verification (Kiểm tra trạng thái):* Dùng các assertion như `assertEquals`, `assertThat` để kiểm tra giá trị trả về của phương thức hoặc trạng thái nội tại của đối tượng sau khi thực thi.
     - *Behavior Verification (Kiểm tra tương tác hành vi):* Sử dụng `Mockito.verify(repo, times(1)).save(...)` hoặc `verifyNoInteractions(...)` làm checkpoint xác nhận rằng đối tượng phụ thuộc có được gọi đúng số lần, đúng thứ tự và đúng tham số hay không (ví dụ: khi từ chối mượn sách thì tuyệt đối không được gọi lệnh lưu phiếu mượn).
3. **Record & Playback (Ghi nhận & Phát lại trong ngữ cảnh Unit Test):**
   - *Diễn giải tương đương và ứng dụng:* Khái niệm Record & Playback khởi nguồn từ kiểm thử UI/E2E (ghi lại thao tác chuột/bàn phím và phát lại). Tuy nhiên ở cấp độ Unit & Backend Component Test, nhóm diễn giải kỹ thuật này theo hai hình thức tương đương:
     - *HTTP Traffic Recording (WireMock / VCR pattern):* Ghi nhận lại request/response từ API bên thứ ba trong lần chạy thật đầu tiên, lưu thành các file JSON stub, và phát lại (replay) cục bộ ở các lần unit test tiếp theo để đảm bảo tính độc lập và tốc độ cao.
     - *Golden Master / Snapshot Testing:* Ghi lại kết quả xuất bản (chụp snapshot cấu trúc dữ liệu hoặc file output) của một hàm phức tạp ở phiên bản ổn định, và so sánh output của phiên bản mới với snapshot đã lưu.
   - *Lý do ưu tiên mã kiểm thử tường minh:* Nhóm không sử dụng công cụ sinh test tự động bằng record thao tác trong case study vì:
     - Test được record thường có tính giòn (fragile), dễ gãy khi cấu trúc code thay đổi nhỏ dù nghiệp vụ không đổi.
     - Thiếu tính rõ ràng về mặt ý đồ nghiệp vụ (lacking business intent): Unit test viết tay theo chuẩn AAA phản ánh trực tiếp tài liệu sống (living specification) của hệ thống, giúp người đọc code hiểu ngay yêu cầu chức năng.

Bốn vòng TDD được tái hiện với test mới fail trước, sau đó sửa logic để toàn bộ test đã thêm pass. Nguồn và log từng vòng được giữ riêng. Đây là bài thực hành hiện tại, chưa có commit lịch sử của 5 thành viên.

2.4. Các trường hợp JUnit / Mockito không xử lý tốt và Anti-patterns

Dù JUnit 5 và Mockito 5 rất mạnh mẽ, việc lạm dụng hoặc dùng sai ngữ cảnh kiểm thử đơn vị dẫn đến nhiều hệ quả tiêu cực (đã được minh họa thực nghiệm trong class `MockitoAntiPatternTest` với tag `@Tag("antipattern")`):

1. **Mock quá sâu (Train Wreck / vi phạm định luật Demeter):**
   - *Biểu hiện:* Chuỗi stub dài dòng `when(a.getB().getC().doSomething()).thenReturn(...)`.

    - *Nguy cơ:* Test bị gắn chặt chẽ vào cấu trúc nội bộ của mạng lưới đối tượng (tight coupling). Bất kỳ tái cấu trúc nhỏ nào (refactoring) cũng làm gãy test dù hành vi tổng thể không đổi.
    - *Khắc phục:* Thiết kế lại hướng đối tượng theo nguyên tắc *Tell, Don't Ask* hoặc đóng gói phương thức trực tiếp trên đối tượng tầng ngoài (`a.doSomethingDirectly()`).
2. **Test tự xác nhận (Self-fulfilling prophecy / Testing the mock):**
    - *Biểu hiện:* Lập trình viên mock một dịch vụ trả về giá trị X, sau đó gọi phương thức mock và assert rằng kết quả đúng bằng X (`when(mock.calc()).thenReturn(10); assertThat(mock.calc()).isEqualTo(10);`).
    - *Nguy cơ:* Test luôn luôn pass 100% nhưng giá trị kiểm thử thực tế bằng 0. Test chỉ đang xác nhận lại cơ chế lưu trữ của Mockito chứ không kiểm thử bất kỳ dòng code nghiệp vụ nào của ứng dụng.
3. **Mock logic thuần túy (Pure Logic / Value Objects):**
    - *Biểu hiện:* Cố tình mock các hàm tính toán toán học hoặc các lớp tiện ích (Utility/Value Object) không hề có I/O hay side-effects.
    - *Nguy cơ:* Làm chậm tốc độ thực thi test đáng kể (do chi phí sinh bytecode dynamic proxy của ByteBuddy), đồng thời che giấu các lỗi tính toán thực tế nếu hàm gốc bị sửa sai.
4. **Hạn chế kỹ thuật khi Mock Static / Final:**
    - Mặc dù Mockito 5 đã kích hoạt sẵn `mock-maker-inline` cho phép mock class final và static method (`mockStatic`), việc này vẫn tạo ra cảnh báo JVM (`Dynamic loading of agents will be disallowed by default in a future release`), làm chậm quá trình khởi động test và là dấu hiệu của kiến trúc ghép nối cứng (tight coupling), khó mở rộng.

3. Code Coverage và chất lượng test

Line coverage đo số dòng thực thi; branch coverage đo các hướng rẽ trong mã đã được chạy. Một dòng có thể chứa nhiều nhánh; vì vậy chỉ xem màu xanh ở dòng là chưa đủ. JaCoCo thu dữ liệu thực thi và tạo HTML cho người đọc, XML/CSV cho xử lý tự động.

3.1. Các loại coverage trên một ví dụ và một bộ test

`FineCalculator` chỉ có điều kiện đơn, nên trên lớp này branch, condition và MC/DC luôn bằng nhau. Để thấy các loại coverage khác nhau, nhóm dùng lớp nhỏ `LoanPeriodPolicy` (số ngày được mượn) và bộ 4 test `LoanPeriodPolicyTest`:

```
public int loanDays(MemberTier tier, boolean newRelease, BigDecimal unpaidFine) {
    int days = 14;
    if (tier == MemberTier.VIP && unpaidFine.signum() == 0) {   // D1 = C1 && C2
        days += 7;
    }
    if (newRelease) {                                           // D2 = C3
        days -= 7;
    }
    return days;
}
```

Lớp có 6 câu lệnh, 2 quyết định (D1, D2), 3 điều kiện con (C1: hạng VIP, C2: không nợ phí, C3: sách mới), 4 đường đi (D1 × D2) và 1 phương thức.

Test	Đầu vào (hạng, sách mới, nợ)	Kỳ vọng	C1 C2 C3	D1 D2
T1	VIP, có, 0	14	T T T	T T
T2	NORMAL, không, 0	14	F T F	F F
T3	VIP, có, 2.000	7	T F T	F T
T4	VIP, không, 0	21	T T F	T F

Bảng dưới cộng dồn từng test. Cột cuối là số JaCoCo đo thật (`evidence/tuan-06/coverage-types/`); các cột khác tính tay theo định nghĩa.

Bộ test	Statement	Decision (branch)	Condition	MC/DC	Path	Method	JaCoCo branch
{T1}	**6/6 (100%)**	2/4 (50%)	3/6	0/3	1/4	**1/1**	3/6 (50%)
{T1, T2}	6/6	**4/4 (100%)**	5/6	2/3	2/4	1/1	5/6 (83%)
{T1, T2, T3}	6/6	4/4	**6/6 (100%)**	**3/3 (100%)**	3/4	1/1	6/6 (100%)
{T1, T2, T3, T4}	6/6	4/4	6/6	3/3	**4/4 (100%)**	1/1	6/6 (100%)

Mỗi test đưa thêm một loại coverage lên 100%:

- **Statement (C0)**: mỗi câu lệnh chạy ít nhất một lần. T1 đi vào cả hai `if` nên đạt ngay 100%, dù chưa lần nào đi nhánh "sai". Đây là lý do statement/line coverage dễ gây ảo giác.
- **Decision/Branch (C1)**: mỗi quyết định nhận cả đúng và sai. T2 bổ sung hai hướng sai.
- **Condition**: mỗi điều kiện con nhận cả T và F. Sau T1, T2, C2 chưa bao giờ sai; T3 (VIP nhưng nợ phí) bổ sung.
- **MC/DC**: mỗi điều kiện con được chứng minh *tự nó* làm đổi kết quả quyết định, các điều kiện khác giữ nguyên. C1: cặp T1/T2 (chỉ đổi hạng, D1 đổi T→F). C2: cặp T1/T3 (chỉ đổi nợ phí). C3: cặp T1/T2 cho D2. MC/DC cần n+1 = 3 test cho D1, trong khi xét mọi tổ hợp (multiple condition coverage) cần $2^2 = 4$ tổ hợp; bộ này vẫn thiếu tổ hợp (NORMAL, có nợ).
- **Path**: mọi đường đi qua hàm. Hai `if` nối tiếp tạo 4 đường; T4 bổ sung đường "được +7 nhưng không bị −7". Số đường tăng theo cấp số nhân với số quyết định, và có thể có đường không khả thi: trong `FineCalculator`, hai biểu thức `tier == VIP ? … : …` luôn cùng kết quả, nên 2 trong 8 đường cú pháp không bao giờ chạy được. Path coverage 100% thường không đạt được trên mã thật.
- **Method**: mỗi phương thức được gọi ít nhất một lần; đạt ngay từ T1, là thước đo thô nhất.

Ví dụ phản chứng condition ≠ decision: chỉ chạy T2 và T3 thì C1, C2, C3 đều đã nhận cả T và F (condition 6/6), nhưng D1 chưa bao giờ đúng, nên dòng `days += 7` không chạy (statement 5/6, decision 3/4). JaCoCo đo bộ này được 6/7 dòng, 5/6 nhánh (`coverage-types/alt-t2-t3/`). Vì vậy condition coverage không suy ra branch coverage, và ngược lại.

3.2. Coverage gate, test bẫy và mutation (CC-1 → CC-4)

Trên riêng `FineCalculator`, bộ baseline 1 test phủ 5/10 nhánh (50%); bộ đầy đủ phủ 10/10 nhánh (100%). Gate dùng `jacoco:check` yêu cầu ít nhất 90% branch cho lớp này. Chạy baseline và bật gate làm Maven fail; bộ đầy đủ giữ nguyên gate thì pass. Các con số này không phải độ phủ 100% của toàn ứng dụng.

Bộ test bẫy đi qua đủ nhánh nhưng không assert số tiền của trường hợp hợp lệ. Nó vẫn đạt 100% branch. PIT tạo 11 mutation: test yếu diệt 3; bộ đầy đủ diệt 10. Một mutant còn sống đổi `daysLate <= graceDays` thành <; tại đúng ngưỡng, cả hai cách vẫn trả 0 nên tương đương theo đặc tả hiện tại. Không sửa test để ép diệt một mutant không đổi hành vi quan sát được.

Kết quả cho thấy coverage cần được kết hợp với assertion có ý nghĩa và dữ liệu biên. Mutation testing hỗ trợ tìm test yếu nhưng có chi phí chạy thêm và cần đọc mutant còn sống.

3.3. JaCoCo đo được gì, không đo được gì

JaCoCo gắn probe vào **bytecode** lúc chạy (Java agent), không đọc mã nguồn. Báo cáo có 6 counter: instructions, branches, lines, methods, classes và cyclomatic complexity (tài liệu counters). Đối chiếu với các loại coverage ở mục 3.1:

Loại coverage	JaCoCo	Giải thích, dẫn chứng trên `LoanPeriodPolicy`
Statement	Có (gián tiếp)	Counter Instructions (17 lệnh bytecode) và Lines (7 dòng) thay cho câu lệnh nguồn. Một dòng có thể chỉ phủ một phần: JaCoCo tô vàng.
Method	Có	Counter Methods tính cả constructor mặc định do trình biên dịch sinh, nên lớp có 1 phương thức viết tay nhưng JaCoCo báo 2/2.
Decision (branch)	Gần đúng, không trùng	JaCoCo đếm mỗi lệnh nhảy có điều kiện trong bytecode. `C1 && C2` biên dịch thành 2 lệnh nhảy nên D1 có 4 nhánh, cả lớp 6 nhánh, không phải 4 hướng quyết định. Với {T1, T2}, decision đã 100% nhưng JaCoCo báo 5/6 (83%).
Condition	Gần đúng (có short-circuit)	Vì đếm từng lệnh nhảy, branch của JaCoCo gần với condition coverage *có tính short-circuit*: điều kiện không được đánh giá thì không tính. Bộ T2 + T3 phủ đủ giá trị T/F của C1–C3 nhưng JaCoCo vẫn báo 5/6, vì C2 chỉ được đánh giá khi C1 đúng.

Loại coverage	JaCoCo	Giải thích, dẫn chứng trên `LoanPeriodPolicy`
MC/DC	Không	Không có khái niệm cặp test chứng minh từng điều kiện độc lập. Riêng chuỗi `&&` ngắn như D1, muốn đủ 6/6 nhánh JaCoCo thì cũng phải có đúng các cặp MC/DC ({T1, T2, T3}). Điều này không được đảm bảo với biểu thức phức tạp hơn, và JaCoCo không báo cặp nào còn thiếu.
Path	Không	Bước 3 và bước 4 cùng 6/6 nhánh, dù T4 là đường đi mới. JaCoCo không biết thứ tự các nhánh trong một lần chạy.
Nhánh trong thư viện	Không	Chỉ đo lớp của dự án. Quyết định nằm trong `Math.min` của `FineCalculator` (mức trần 50.000 VND) không có nhánh nào trong báo cáo, xem mục 3.4.
Chất lượng assertion	Không	Test bẫy CC-4 không assert số tiền vẫn đạt 100% branch. Cần PIT hoặc review.

Các điểm cần nhớ khi đọc báo cáo JaCoCo:

- **Ngoại lệ không được tính là nhánh.** Khối `try/catch` không làm tăng số nhánh. Theo tài liệu JaCoCo, dòng ném ngoại lệ giữa chừng có thể bị tô là chưa chạy vì probe nằm sau lệnh.
- **Mã do trình biên dịch sinh** (constructor mặc định, phương thức của `enum`, `record`…) có thể làm số method/instruction khác với mã nguồn. JaCoCo có bộ lọc cho một số trường hợp, nhưng không lọc tất cả.
- **"Branch 100%" không có nghĩa là "decision 100%".** Hai khái niệm khác cách đếm. Khi so ngưỡng gate (ví dụ 90% branch) cần ghi rõ là branch theo JaCoCo.

3.4. Các trường hợp JaCoCo và PIT không xử lý tốt

Bốn tình huống dưới đây đều chạy thật trên case study. Số đo nằm ở `evidence/tuan-05/` (CC-4) và `evidence/tuan-06/coverage-blindspot/`, `evidence/tuan-06/pit-cost/`.

1. Coverage 100% nhưng test yếu (CC-4). `FineCalculatorTrapTest` gọi mọi nhánh nhưng không assert số tiền. JaCoCo báo 100% branch, gate pass. Chỉ PIT mới lộ ra: test này diệt được 3/11 mutant, bộ đầy đủ diệt 10/11. Coverage chỉ cho biết code *đã chạy*, không cho biết kết quả *đã được kiểm*.

2. Quyết định ẩn trong thư viện: cả JaCoCo lẫn PIT mặc định đều không thấy. Mức trần 50.000 VND của `FineCalculator` nằm trong `Math.min(amount, 50_000L)`. Đây là một quyết định, nhưng nó nằm trong JDK nên không có nhánh nào trong bytecode của lớp. `FineCalculatorNoCapTest` assert đúng 4 số tiền và 2 ngoại lệ, nhưng không ca nào vượt trần:

Lượt chạy	Kết quả
JaCoCo, bộ no-cap	10/10 nhánh (100%), gate 90% **pass**
PIT mặc định, bộ no-cap	10/11 mutant bị diệt (91%), **bằng đúng** bộ đầy đủ

Lượt chạy	Kết quả
PIT `-Dmutators=DEFAULTS,INLINE_CONSTS`, bộ no-cap	13/16; mutant `50000 → 50001` **sống**
PIT `-Dmutators=DEFAULTS,INLINE_CONSTS`, bộ đầy đủ	15/16; mutant hằng số bị diệt nhờ các ca vượt trần trong CSV (NORMAL 26 ngày, VIP 53 ngày, `Integer.MAX_VALUE`)

Nghĩa là nếu ai đó xóa luật trần, bộ no-cap vẫn xanh, coverage vẫn 100%, điểm PIT mặc định vẫn y như bộ tốt. Chỉ khi bật thêm mutator hằng số (không nằm trong bộ mặc định) hoặc đọc đặc tả để thêm ca biên, lỗ hổng mới lộ ra. Cũng trong lượt này, mutant "bỏ điều kiện `daysLate <= graceDays`" sống vì bộ no-cap không có ca VIP trễ dưới thời gian ân hạn (ví dụ 1 ngày).

3. Mutant tương đương: PIT không tự kết luận được. Ở mọi lượt chạy, mutant đổi `daysLate <= graceDays` thành `<` ở dòng 15 đều sống. Tại đúng ngưỡng (`daysLate == graceDays`), code gốc trả `0` ngay. Bản mutant đi tiếp và tính `(graceDays − graceDays) × rate`, cũng ra `0`. Không có đầu vào nào phân biệt được hai bản, nên không test nào diệt được mutant này. PIT vẫn tính nó vào phần "sống" (điểm tối đa của bộ đầy đủ là 10/11 chứ không phải 11/11). Người review phải tự chứng minh mutant tương đương; không nên viết test vô nghĩa để "ép" điểm 100%.

4. PIT chậm hơn nhiều so với chạy test.

Phạm vi	Chạy test thường	PIT	Ghi chú
`FineCalculator` (3 lượt)	test 0,6 s; cả Maven trung vị 16,1 s	PIT tự báo 4 s; cả Maven trung vị 18,9 s	11 mutant, khoảng 3 test/mutant
Toàn project (1 lượt)	39 test, cả Maven 22,3 s	PIT 75 s (coverage 10 s + mutation 65 s); cả Maven 84,9 s	46 mutant, 72 lượt chạy test

Với lớp nhỏ, thời gian khởi động Maven che mất chi phí của PIT. Khi mở ra cả project, PIT chậm gấp khoảng 4 lần lượt chạy test thường, dù project chỉ có vài lớp: mỗi mutant phải chạy lại các test phủ nó, kể cả test Spring khởi tạo context. Chi phí tăng theo số mutant (tỉ lệ với lượng mã) nhân số test phủ mỗi mutant. Cách giảm: giới hạn `targetClasses` cho phần logic quan trọng (cấu hình hiện tại chỉ chọn `FineCalculator`), tăng `threads`, dùng phân tích tăng dần (`withHistory`), hoặc chạy PIT theo lịch thay vì ở mọi PR. Lượt toàn project cũng cho thấy 4 mutant `NO_COVERAGE` ở getter của `LoanEntity` và 1 mutant sống ở `MemberEntity.id`: các test persistence không đọc các trường đó.

4. CI/CD

CI tự động kiểm tra các thay đổi được tích hợp thường xuyên. Continuous Delivery tạo sản phẩm sẵn sàng phát hành với bước quyết định triển khai; Continuous Deployment tự động đưa thay đổi đạt yêu cầu lên môi trường đích. Bản case study hiện dừng ở kiểm thử và tạo JAR.

Workflow gồm `test` chạy JDK 25, `coverage-gate` phụ thuộc kết quả test, `package` phụ thuộc gate. Cache giữ dependency Maven; artifact lưu test report, coverage và JAR; summary giúp người xem đọc điều kiện gate. Jenkinsfile mô tả cùng thứ tự công việc bằng cú pháp pipeline của Jenkins.

Workflow đã qua actionlint, các lệnh Maven đã chạy tại máy. Tài khoản GitHub đang dùng chỉ có quyền READ trên repo nhóm. Do đó chưa có lần chạy Actions, PR đỏ/xanh hay xác nhận required checks chặn merge. Jenkinsfile chưa được chạy trên Jenkins. Việc upload cấu hình không tự bật quy tắc bảo vệ nhánh; người quản trị cần cấu hình required status checks.

Phép đo JaCoCo trên bản Java 25 có ba lượt/cấu hình: trung vị không bật JaCoCo là 9,727 giây; có JaCoCo/report/gate là 12,754 giây. Phép đo repo Maven lạnh/ấm đã đủ ba cặp hợp lệ trên Java 25, trung vị 48,412/12,539 giây. Cặp bổ sung chạy sau hai cặp đầu, mất 479,523/11,940 giây; dùng kho trống cho lượt lạnh và dùng lại kho đó cho lượt ấm, ưu tiên IPv4 và tăng thời gian chờ tải. Điều kiện mạng thay đổi nên không suy ra mức tăng tốc cố định từ trung vị. Log tải lỗi được giữ riêng và không tính vào số liệu hoàn thành. Phép đo cache dependency tại máy không đồng nhất với thời gian restore cache trên GitHub hoặc Jenkins.

5. Khảo sát và lựa chọn công cụ

Mục tiêu	Công cụ thực hành	Công cụ đối chiếu	Cơ sở nhận xét
Viết/chạy test Java	JUnit Jupiter, Mockito, AssertJ	TestNG, Spock, JUnit 4	Chọn bộ do Spring Boot quản lý; đã chạy test trong case study. Chưa benchmark các bộ đối chiếu.
Đo độ phủ	JaCoCo	OpenClover (đã chạy), Cobertura, coverage IDE	Đã chạy HTML/XML và gate với JaCoCo; OpenClover chạy trên cùng bộ test, so sánh ở mục 5.2. Cobertura và coverage IDE chưa chạy.
Kiểm chất lượng assertion	PIT	Đọc test thủ công	Đã chạy cùng lớp với test mạnh/yếu; phải phân tích mutant tương đương.
Tự động kiểm tra	GitHub Actions	Jenkins, GitLab CI, CircleCI	Có workflow và Jenkinsfile; mới kiểm tra tĩnh/lệnh tại máy, chưa đo trên dịch vụ.

5.1. So sánh chuyên sâu: JUnit 5 (Jupiter) vs TestNG

Trong hệ sinh thái Java, **JUnit 5** và **TestNG** là hai framework kiểm thử phổ biến nhất. Dưới đây là bảng so sánh chi tiết trên 5 tiêu chí kỹ thuật:

Tiêu chí	JUnit 5 (Jupiter)	TestNG	Đánh giá & Khuyến nghị
1. Data-Driven Testing	Hỗ trợ đa dạng qua `@ParameterizedTest`: `@ValueSource`, `@CsvFileSource`, `@MethodSource`, `@ArgumentsSource`. Cực kỳ trực quan khi đọc file CSV bên ngoài (`fine-cases.csv`).	Sử dụng `@DataProvider` trả về mảng 2 chiều (`Object[][]`) hoặc `Iterator<Object[]>`. Cần viết code Java đọc CSV thủ công nếu dùng file ngoài.	**JUnit 5 thắng thế** về độ tiện lợi khi tích hợp file CSV/dữ liệu ngoài; TestNG mạnh khi dữ liệu sinh động bằng mã code.

Tiêu chí	JUnit 5 (Jupiter)	TestNG	Đánh giá & Khuyến nghị
2. Phân nhóm & Phụ thuộc test	Dùng `@Tag` để lọc test trong build tool (`excludedGroups`). Không hỗ trợ phụ thuộc (dependency) giữa các test theo thiết kế (tuân thủ nguyên tắc độc lập FIRST).	Hỗ trợ cực mạnh với `groups`, `dependsOnGroups`, `dependsOnMethods`. Cho phép bỏ qua test downstream nếu test upstream fail.	**TestNG linh hoạt hơn** cho Integration/E2E test có thứ tự; **JUnit 5 chuẩn mực hơn** cho Unit test (tránh test phụ thuộc nhau).
3. Chạy song song (Parallel execution)	Cấu hình qua file `junit-platform.properties` (`junit.jupiter.execution.parallel.enabled=true`). Hỗ trợ đồng bộ tài nguyên qua `@ResourceLock`.	Cấu hình trực tiếp trong file XML suite (`<suite parallel="methods" thread-count="4">`) hoặc qua thuộc tính `@Test(threadPoolSize = 3)`.	Cả hai đều rất mạnh; **TestNG cấu hình qua XML dễ kiểm soát** theo từng bộ test suite hơn.
4. Kiến trúc mở rộng (Extensibility)	Kiến trúc mô-đun: JUnit Platform + JUnit Jupiter + JUnit Vintage. Cơ chế mở rộng duy nhất, mạnh mẽ: `Extension API` (`@ExtendWith`).	Dùng hệ thống Listener đa dạng (`ITestListener`, `ISuiteListener`, `IInvokedMethodListener`) kế thừa từ interface.	**JUnit 5 hiện đại hơn**, loại bỏ hoàn toàn mâu thuẫn giữa Runner và Rule của JUnit 4.
5. Tích hợp Spring Boot & Hệ sinh thái	Được đóng gói mặc định trong `spring-boot-starter-test`. Tích hợp hoàn hảo với Spring TestContext Framework (`@SpringBootTest`, `@WebMvcTest`, `@MockitoBean`).	Cần loại trừ JUnit và cấu hình riêng thư viện TestNG + `spring-boot-starter-test`. Cần kế thừa `AbstractTestNGSpringContextTests`.	**JUnit 5 là tiêu chuẩn mặc định** của ngành cho các ứng dụng Spring Boot hiện đại.

Kết luận lựa chọn: Nhóm chọn **JUnit 5** làm công cụ thực hành chính vì:

- Là chuẩn mặc định của Spring Boot 4.x và Java 25.
- Cú pháp `@ParameterizedTest` với file CSV phản ánh trực tiếp kỹ thuật Data-driven mà không cần viết mã đọc file trung gian.
- Tuân thủ nghiêm ngặt nguyên tắc FIRST, ngăn chặn tình trạng viết test phụ thuộc thứ tự (anti-pattern thường gặp ở TestNG).

5.2. So sánh công cụ coverage: JaCoCo vs OpenClover

Nhóm chạy thử OpenClover 4.5.2 trên bản sao `case-study/` bằng lệnh gọi plugin đầy đủ tọa độ (không sửa `pom.xml`), cùng bộ test T1–T4 và cả bộ 39 test. Cả 39 test đều pass trên JDK 25. Số đo nằm ở `evidence/tuan-06/openclover/`.

Tiêu chí	JaCoCo 0.8.15	OpenClover 4.5.2	Nhận xét
1. Cách đo	Java agent gắn probe vào **bytecode** khi nạp lớp; không đụng mã nguồn.	**Sửa mã nguồn** (instrument) sang thư mục riêng rồi biên dịch lại; ghi dữ liệu vào `clover.db`.	JaCoCo đo được cả lớp không có mã nguồn. OpenClover bám sát cú pháp nguồn nhưng phụ thuộc parser hỗ trợ phiên bản Java.

Tiêu chí	JaCoCo 0.8.15	OpenClover 4.5.2	Nhận xét
2. Đơn vị "nhánh"	Lệnh nhảy bytecode: `LoanPeriodPolicy` có 6 nhánh.	Kết quả đúng/sai của **cả biểu thức điều kiện** (decision): 4 conditional.	Cùng bộ {T1, T2}: JaCoCo 5/6 (83%), OpenClover 4/4 (100%). Bộ T2 + T3: JaCoCo 5/6, OpenClover 3/4. Hai con số "branch" không so trực tiếp được.
3. Statement / method	Instructions 17, lines 7, methods 2 (tính cả constructor mặc định).	Statements 6, methods 1 (chỉ phương thức viết trong nguồn).	OpenClover gần với định nghĩa statement/method trong giáo trình hơn.
4. Chỉ số tổng hợp	Từng counter riêng; gate `jacoco:check` theo counter (nhóm dùng BRANCH ≥ 90% cho `FineCalculator`).	Total Percentage Coverage (TPC) gộp statement + conditional + method; gate `clover:check` theo TPC.	Ngưỡng 90% ở hai công cụ không cùng ý nghĩa; khi đổi công cụ phải đặt lại ngưỡng.
5. Thời gian (3 lượt, cả bộ test)	`clean verify` (test + report + gate): 37,1 / 46,6 / 47,9 s, trung vị **46,6 s**.	`clean clover:setup test clover:clover` (tắt JaCoCo): 38,8 / 44,8 / 43,4 s, trung vị **43,4 s**.	Chênh lệch nhỏ hơn dao động giữa các lượt trên cùng máy, nên không kết luận công cụ nào nhanh hơn với dự án nhỏ này.
6. Hệ sinh thái	Mặc định trong hầu hết hướng dẫn Maven/Gradle, SonarQube, Codecov; cập nhật đều theo phiên bản JDK.	Mã nguồn mở từ Atlassian Clover; bản 4.5.2 phát hành 01/2024. Ưu điểm theo tài liệu: coverage theo từng test, test optimization (chỉ chạy test liên quan).	Nhóm chưa thử tính năng coverage theo từng test của OpenClover.

Cobertura không được chạy thử: bản cuối 2.1.1 phát hành năm 2015 và không còn được phát triển, nên nhóm không đưa vào so sánh thực nghiệm.

Kết luận: nhóm giữ **JaCoCo** cho case study và CI vì không phải sửa mã nguồn, có sẵn gate theo counter và được tích hợp rộng. OpenClover hữu ích để minh họa vì đo theo decision ở mức mã nguồn, cho thấy rõ "branch" của mỗi công cụ là một định nghĩa khác nhau.

Các bảng khảo sát ở gói đề xuất là dữ liệu tham khảo ban đầu, không thay cho kết quả trải nghiệm. Cần kiểm lại tình trạng phát triển, giấy phép, hạn mức và giá tại thời điểm nộp nếu đưa các thông tin đó vào bản chính thức. Mockito 5 đã dùng inline mock maker mặc định; không giữ kết luận cũ rằng mock final/static luôn cần thêm module.

6. Áp dụng AI

AI-1 sinh bản nháp 3 test; các test chạy đúng nhưng chỉ phủ 70% nhánh. AI-2 đọc JSON trích từ JaCoCo và source, nhận ra thiếu null, ngày âm, nhánh tính phí VIP; sinh 3 test bổ sung. Profile riêng cho phép đo đúng bộ 3 → 6 test mà không đưa bộ tham chiếu vào kết quả.

AI-3 có prompt và client gọi Gemini qua biến môi trường, cùng workflow lưu artifact. Chế độ chuẩn bị không gọi dịch vụ và ghi `api_called=false`. Trong Tuần 06 đã có một lần gọi live bằng `gemini-3.6-flash` trên diff fixture; sau HTTP 503, retry thành công và lưu 4 findings. Đối chiếu với expected fixture

đạt 4 đúng, 0 thừa, 0 bỏ sót. Fixture có lỗi seed nên đây là kiểm tra luồng gọi API, không phải benchmark độc lập hay review toàn diện repository.

AI-4 có 13 dòng dữ liệu hợp lệ, được chạy với calculator thật bằng profile riêng. Bộ tham chiếu có 15 dòng; hai bộ trùng 13 ca, bộ AI không có NORMAL 3 ngày và VIP 0 ngày. Dữ liệu sai null/ngày âm được kiểm riêng bằng JUnit. Hai bộ đều có AI hỗ trợ chuẩn bị, nên cần thành viên nhóm tự viết/rà soát bộ chuẩn để có so sánh con người–AI hợp lệ.

Tất cả nội dung AI cần được review theo checklist: đúng nghiệp vụ, assertion cụ thể, không bịa API, không mock logic thuần, đọc số đo thực tế. Nhóm sẽ bổ sung người xác nhận; chưa quy đóng góp thực tế cho TV1–TV5 từ sản phẩm do trợ lý chuẩn bị.

7. Bài tập áp dụng

Bộ bài tập và đáp án gợi ý được chuẩn hóa tại BaiTap_Tuan_06.md (`docs/thuc-hanh/BaiTap_Tuan_06.md`): mở rộng data-driven Unit Test, phân biệt coverage với sức assertion qua PIT, đọc coverage gate trong pipeline, và chọn mức integration phù hợp. Ví dụ `STUDENT` là chính sách giả định riêng của bài tập; không sửa hợp đồng `NORMAL`/`VIP` trong source seminar.

8. Gói seminar và trạng thái hoàn thiện

Gói có source, bộ test, log trước/sau, HTML/XML coverage, PIT, replay TDD, script AI, báo cáo chi tiết, slide, bài tập và hướng dẫn tái hiện. Ba video theo từng chủ đề có kịch bản trong `KichBan_QuayVideo_Tuan_06.md`; chưa có video đã quay. Bảng `BangCongViec.md` ghi giới hạn còn lại của GitHub/Jenkins, video và xác nhận của thành viên.

Số đo và đường dẫn: xem minh chứng (`evidence/tuan-05/README.md`) và minh chứng Tuần 06 (`evidence/tuan-06/README.md`). Hướng dẫn tái hiện: HuongDan_Demo.md (`docs/thuc-hanh/HuongDan_Demo.md`).

Nguồn chính thức dùng đối chiếu: JUnit guide, Mockito, JaCoCo, PIT Maven, Spring Boot requirements, GitHub Maven CI, Gemini API.

9. Phần AI, paper và trình bày — Ngô Gia An (23120205)

Phạm vi theo `PhanCong_NhiemVu.md` hiện tại, triển khai với Codex hỗ trợ ngày 28/10/2026. Trưởng nhóm hiện tại là Ngô Gia An; Nguyễn Bảo An phụ trách Unit Test. Các bằng chứng lịch sử ở phần trước không trở thành lượt chạy mới.

9.1. Khả năng tái sử dụng AI trên LoanService

Giữ nguyên nghiệp vụ production. Profile `ai-loan-draft`: 2 test pass, line 14/17 và branch 3/6. Trích gap và đối chiếu source rồi dùng profile `ai-loan-improved`: 10 test pass, line 17/17, branch 6/6. Bộ bổ sung kiểm sách unavailable, member missing, hạn mức 2/3/4, nợ dương 0.01/1000, không ghi khi từ

chối, ID sách/thành viên và Clock.fixed bằng ArgumentCaptor. PIT trên bộ này diệt 9/9 mutant trong cấu hình hiện tại; không chứng minh mọi lỗi đều được tìm thấy. Bộ chuẩn repo vẫn 39/39, gate pass trên JDK 25.0.5.

Profile `ai-loan-mistake` cố ý cài kỳ vọng sai "đang giữ 3 sách vẫn mượn được", fail đúng một assertion. Fixture dùng để dạy kiểm oracle, không phải lỗi quan sát từ API hay kết quả paper. Prompt/source/log: AI-5 review `(ai/results/AI-5-loan-review.md)`, evidence `(evidence/gia-an/README.md)`.

9.2. AI-4 đối chứng độc lập

Gia An xác nhận chưa có dữ liệu người viết. Đã tạo quy trình `(ai/human/README.md)`, mẫu rỗng và script freeze/compare. Người chưa xem dữ liệu AI tự viết oracle theo đặc tả, khóa CSV bằng SHA-256/commit trước khi mở output AI; so ca trùng/thiếu/thêm và kết quả khác rồi thực thi profile `ai-human`. Test công cụ bằng fixture synthetic không thay cho đối chứng độc lập. Chưa có số liệu đánh giá người-với-AI.

9.3. Hai paper có source và kiểm hạng

Danh mục có DOI, năm, source, revision, nguồn hạng và giới hạn: research/papers.json `(research/papers.json)`; phân tích và lệnh chạy `(research/README.md)`.

- **CoverUp: Effective High Coverage Test Generation for Python**, Pizzorno và Berger, FSE 2025 Research Papers / PACMSE, DOI 10.1145/3729398. Kết hợp coverage với ngữ cảnh và vòng phản hồi cho LLM; dùng tool Python gốc.
- **ChatUniTest: A Framework for LLM-Based Test Generation**, Chen và cộng sự, FSE 2024 Demonstrations, DOI 10.1145/3663529.3663801. Dùng adaptive focal context và generation–validation–repair cho Java; giữ source gốc và trial plugin hiện tại tách riêng.

CORE2023 chính thức ghi FSE A*. Đây là hạng venue; ChatUniTest thuộc demo track cần xác nhận GV chấp nhận. Hai năm xuất bản 2024–2025 nằm trong yêu cầu 2021–2026.

Thử nghiệm	Kết quả thật	Giới hạn
CoverUp script gốc phân tích 163 module MT	overall line+branch 89,0%, MuTAP gpt4o_zero 76,8%; khớp 89% vs 77% làm tròn ở abstract v4	Dữ liệu tác giả công bố, không sinh lại test; dependencies phân tích mới có khóa phiên bản
CoverUp 0.6.0 trên `scripts/coverage_gaps.py`	dry-run pass, coverage smoke ban đầu 23,6%, sau dry-run vẫn 23,6%	Không gọi model, không tạo test mới, không dùng làm kết quả generation
ChatUniTest plugin 2.1.1 trên bản sao case study	parse pass: 20 class, 28 method	Chỉ phân tích source, endpoint localhost không có model; chưa generation hoặc tái lập experiment gốc

Không tự implement lại thuật toán paper. Còn thiếu lượt model thật/benchmark và thử test sinh ra trên source nhóm cho cả hai bài; không đánh dấu mục E hoàn thành. Chia lượt chạy cho thành viên là kế hoạch đề xuất trong research, chưa khẳng định họ đã nhận/chạy.

9.4. Slide, khai báo AI và video

Đã hợp nhất nội dung phần Gia An vào slide nền trắng/chữ đen; nguồn JSON sinh HTML và Markdown đồng bộ. Danh mục video `(docs/gia-an/Video_Artifacts.md)` ghi từng GA-01..07, nội dung, input/output và tình trạng. Slide nhắc video Unit Test/Coverage/CI/Case study theo người phụ trách; chờ URL thật. Video bắt buộc theo phân công hiện tại, nếu gộp phải có timestamp thật; chưa quay/đăng, không điền link giả.

Khai báo Gia An `(ai-usage-declaration/01_NgoGiaAn.md)` ghi đúng phần Codex tạo và chạy; phần người tự review/ký còn chờ. Điều phối `(docs/gia-an/DieuPhoi.md)` chờ ngày hạn thật để tính mốc gửi slide/gặp GV trước 3–4 ngày, hai buổi tập và các mục nộp. Chưa thực hiện cuộc họp, liên hệ GV, điền Google Sheet hay nộp bài thay người dùng.